

Multi-Courier Integration Platform

Design Document

1. Overview

This service provides a unified backend API for creating, tracking, cancelling, and bulk-processing shipments across multiple courier partners.

The first real courier integration is UrbaneBolt. The design also includes MockCourier as a second adapter to demonstrate that new couriers can be added without changing the consumer-facing API.

Internal consumers, such as an order management system or frontend, call one normalized API and pass a `courier_partner` identifier. They do not need to know the request format, authentication flow, status codes, or error shape of any specific courier.

2. Architecture

None

Consumer API

|

v

Controllers and Validators

|

v

Services

|

v

Courier Registry

|

v

Courier Adapters

|

v

External Courier APIs

Responsibilities are separated as follows:

Layer	Responsibility
Routes and controllers	HTTP request handling, response status codes, and routing
Validators	Field-level request validation using the normalized schema
Services	Business rules, idempotency, persistence, batch orchestration, and error mapping
Repositories	PostgreSQL reads/writes using parameterized SQL
Courier registry	Resolves <code>courier_partner</code> to the correct adapter
Courier adapters	Partner-specific authentication, payload mapping, status mapping, and HTTP calls

3. Pluggable Courier Design

The core design uses the **Adapter Pattern** with a lightweight **Registry/Factory**.

Every courier implements the same internal interface:

- `authenticate()`
- `createOrder()`

- `trackShipment()`
- `cancelOrder()`

Current adapters:

Adapter	Purpose
<code>UrbaneBoltAdapter</code>	Real UrbaneBolt UAT integration
<code>MockCourierAdapter</code>	In-process test/demo courier
<code>GenericRestCourierAdapter</code>	Config-driven adapter for simple JSON REST couriers

Adding a new coded courier requires a new adapter folder and one registry definition. It does not require changes to controllers, routes, unified DTOs, existing adapters, or service logic.

For straightforward REST couriers, the admin API/UI can register a new courier through configuration only. The repository also includes `scripts/demo-courier-server.ts`, which starts a small local fixture courier so reviewers can verify no-code onboarding without another real carrier account.

4. Database Design

PostgreSQL is used as the persistence layer. The main tables are:

Table	Purpose
<code>courier_partners</code>	Registered couriers, enabled flag, adapter kind, and JSON config
<code>orders</code>	Internal order state, courier IDs, AWB, status, request/response audit payloads, and timestamps
<code>tracking_events</code>	Append-only tracking history with timestamp and raw courier payload

Table	Purpose
<code>batches</code>	Bulk request status and success/failure counters
<code>batch_items</code>	Per-item bulk results, including items that fail before an order row exists

Key design choices:

- `orders.order_id` is unique and acts as the idempotency key.
- Tracking history is append-only.
- Order status updates and tracking-event inserts happen in a transaction.
- Status values are constrained at the database level.
- Bulk results are stored separately in `batch_items` so every submitted item can be reported, even when no `orders` row is created.

Schema changes are versioned under:

`src/db/migrations`

Pending migrations are applied automatically on application startup. They can also be run explicitly:

`npm run migrate`

5. Bulk Processing

The bulk endpoint accepts up to 100 orders:

`POST /api/v1/orders/bulk`

The endpoint returns immediately with HTTP 202 and a `batch_id`. Orders are then processed in the background with bounded concurrency.

This approach keeps the API responsive and avoids making one HTTP request wait for up to 100 courier calls. Each item in the batch can use a different courier partner.

Batch status and per-item results are available through:

`GET /api/v1/orders/bulk/:batch_id`

Supported bulk behavior:

- partial success and failure
- duplicate detection inside the same batch request
- idempotent replays of existing `order_id` values
- per-item error reporting
- background processing with configurable concurrency

6. Error Handling

All endpoints use one normalized error response:

```
JSON
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable message",
    "details": {}
  },
  "request_id": "..."
```

Error handling rules:

Scenario	Behavior
Bad input	HTTP 400 with field-level validation details
Unknown courier	HTTP 400 with supported courier list

Scenario	Behavior
Disabled courier	HTTP 400 with normalized error code
Courier 4xx	Mapped to platform error codes without leaking raw courier response
Courier 5xx, timeout, or network failure	Retry with configurable backoff, then persist failure
Courier auth expiry	Re-authenticate and retry once where supported

Failures are logged with order id, courier partner, request id, error type, and stack trace where available.

7. Configuration And Seeding

Server configuration is loaded from environment variables and validated at startup.

Important configuration groups:

Config	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection
<code>ADMIN_API_TOKEN</code>	Protects admin courier configuration APIs
<code>BULK_MAX_ORDERS</code>	Maximum orders accepted by the bulk endpoint
<code>BULK_CONCURRENCY</code>	Number of concurrent background courier calls
<code>CORS_ALLOWED_ORIGINS</code>	Allowed origins in production
<code>RATE_LIMIT_*</code>	API rate limit settings

Config	Purpose
<code>URBANEOLT_*</code>	Optional local/UAT seed values for UrbaneBolt

Built-in courier rows are seeded on startup for:

- `urbanebolt`
- `mockcourier`

Seed data can also be applied explicitly:

```
npm run seed
```

The seeder is idempotent. It creates missing built-in courier rows and fills blank config values from environment defaults without overwriting values already set by an admin.

Runtime courier config lives in `courier_partners.config`. Secret fields are masked in admin API responses.

8. Trade-Offs

Area	Decision	Trade-off
Database access	Raw <code>pg</code> with SQL migrations	Explicit and lightweight, but less tooling than a full migration framework
Bulk worker	In-process background processing	Simple to run locally, but not durable across process restarts

Area	Decision	Trade-off
Generic courier onboarding	JSON REST configuration	Covers common REST APIs, but unusual protocols still need coded adapters
Admin access	Single bearer token	Suitable for assignment/demo use, but production should use users, roles, and audit trails

9. Production Considerations

For a production deployment, the first improvements would be:

- move bulk processing to a durable queue such as SQS, BullMQ, or another job runner
- add consumer API authentication for `/api/v1/orders`
- use a managed secrets store for courier credentials
- add richer admin user management and audit logs
- expand courier webhook support for real-time tracking updates